UNIVERSIDADE FEDERAL DE PERNAMBUCO
CENTRO DE INFORMÁTICA

INFRA-ESTRUTURA DE HARDWARE — IF674

Relatório de Projeto

Processador multiciclo de 32 bits — 2026.2

Equipe EQ01

Carlos Henrique Gonçalves dos Santos (CC) — `chgs3`

Robson Thiago Duarte (EC) — `rtds`

Recife — 21/10/2026

Sumário

1 Introdução **1**
1.1 Escopo e critérios do projeto . . . 1
1.2 Situação da implementação e da evidência . . . 1
1.3 Relação com as aulas . . . 1

2 Unidade de processamento **2**
2.1 Visão geral do caminho de dados . . . 2
2.2 Formatos, busca e acesso à memória . . . 4
2.3 Grupos de reaproveitamento . . . 5
2.4 Pilha e extensão da arquitetura . . . 5
2.5 Demais operações e cuidados de integração . . . 6

3 Descrição das entidades **6**
3.1 Componentes recebidos na parte 3 . . . 6
3.2 Registradores e banco . . . 7
3.3 ALU, deslocador e unidades especiais . . . 7
3.4 Memória e tratamento de bytes . . . 7
3.5 Unidade de controle e seletores . . . 7

4 Descrição dos estados do controle **7**
4.1 Como ler o mapa de estados . . . 7
4.2 Estados compartilhados . . . 8
4.3 Sequência de `push` e `pop` . . . 8
4.4 Exemplo de execução e bytes . . . 9
4.5 Exceções e retorno . . . 9

5 Conjunto de simulações **11**
5.1 Programas de apoio recebidos na parte 3 . . . 11
5.2 Convenção de preenchimento . . . 11
5.3 Casos prioritários da pilha . . . 12
5.4 Instruções do formato R . . . 12
5.5 add . . . 12
5.6 and . . . 12
5.7 div . . . 13
5.8 mult . . . 13
5.9 jr . . . 14
5.10 mfhi . . . 14
5.11 mflo . . . 14
5.12 sll . . . 15
5.13 slt . . . 15
5.14 sra . . . 16
5.15 sub . . . 16

5.16 push . . . 16
5.17 pop . . . 17
5.18 Instruções do formato I . . . 17
5.19 addi . . . 17
5.20 beq . . . 18
5.21 bne . . . 18
5.22 lb . . . 19
5.23 lui . . . 19
5.24 lw . . . 19
5.25 sb . . . 20
5.26 sw . . . 20
5.27 Instruções do formato J . . . 21
5.28 j . . . 21
5.29 jal . . . 21
5.30 Simulações isoladas das entidades implementadas . . . 21
5.30.1 Unidade de controle . . . 22
5.30.2 Multiplicador . . . 22
5.30.3 Divisor . . . 22
5.30.4 Outras entidades próprias . . . 22

6 Conclusão **22**

A Catálogo das 23 instruções **23**

B Decisões a confirmar antes da entrega **24**

Lista de Figuras

1 Fluxos funcionais da CPU multiciclo. Cada linha resume uma sequência de estados; a UC configura os caminhos a cada ciclo. . . . 2
2 Circuito completo do processador multiciclo exportado do FigJam. . . . 3
3 Detalhe do circuito: memória, IR, banco, ALU, EPC e seletores próximos. . . . 4
4 Resumo lógico da transição entre busca, decodificação e famílias de execução. . . . 8
5 Legenda de grupos e sinais de controle usada no mapa de estados da equipe. . . . 8
6 Visão geral do mapa de estados e do contexto de controle exportados do FigJam. . . . 10
7 Recorte ampliado do mapa de estados e das transições da UC. . . . 10
8 Simulação de add — inserir captura após validação. . . . 12
9 Simulação de and — inserir captura após validação. . . . 13
10 Simulação de div — inserir captura após validação. . . . 13
11 Simulação de mult — inserir captura após validação. . . . 13
12 Simulação de jr — inserir captura após validação. . . . 14
13 Simulação de mfhi — inserir captura após validação. . . . 14
14 Simulação de mflo — inserir captura após validação. . . . 15
15 Simulação de sll — inserir captura após validação. . . . 15
16 Simulação de slt — inserir captura após validação. . . . 15
17 Simulação de sra — inserir captura após validação. . . . 16
18 Simulação de sub — inserir captura após validação. . . . 16
19 Simulação de push — inserir captura após validação. . . . 17
20 Simulação de pop — inserir captura após validação. . . . 17
21 Simulação de addi — inserir captura após validação. . . . 18
22 Simulação de beq — inserir captura após validação. . . . 18
23 Simulação de bne — inserir captura após validação. . . . 18
24 Simulação de lb — inserir captura após validação. . . . 19
25 Simulação de lui — inserir captura após validação. . . . 19
26 Simulação de lw — inserir captura após validação. . . . 20
27 Simulação de sb — inserir captura após validação. . . . 20
28 Simulação de sw — inserir captura após validação. . . . 20
29 Simulação de j — inserir captura após validação. . . . 21
30 Simulação de jal — inserir captura após validação. . . . 21

Lista de Tabelas

1 Famílias de instruções organizadas pelo caminho predominante. 5
2 Seleções essenciais da extensão da pilha. 5
3 Módulos VHDL e arquivo de memória fornecidos na parte 3. 6
4 Estados de `push` após a captura dos operandos. 8
5 Estados de `pop` após a captura de SP. 9
6 Evolução esperada no exemplo da especificação. 9
7 Programas Assembly fornecidos e finalidade principal. 11
8 Testes mínimos sugeridos para a extensão da pilha. 12
9 Instruções exigidas e efeito arquitetural principal. 23
10 Pendências de integração e validação. 24

1 Introdução

Este relatório descreve a arquitetura e o controle de um processador multiciclo de 32 bits para as 23 instruções definidas na especificação do projeto de IF674 de 2026.2 [1]. O objetivo é associar o comportamento de cada instrução aos blocos do caminho de dados, aos sinais da unidade de controle (UC) e à sequência temporal de estados. A versão 2026.2 substitui `xchg` e `sram` por `push` e `pop`; por isso, o tratamento da pilha e do registrador R29 recebe atenção especial nas Seções 2.4 e 4.3.

Por decisão de Robson Thiago Duarte, as instruções são apresentadas em agrupamentos definidos pela semelhança de seus caminhos de dados e etapas de controle. Essa organização evidencia os recursos compartilhados da arquitetura multiciclo; o catálogo individual das 23 instruções permanece no Apêndice A, e o roteiro de simulação de cada uma está na Seção 5.

O projeto foi organizado a partir da documentação das partes 1, 2 e 3 da equipe [2, 3, 4], do quadro no FigJam [6] e dos conteúdos de instruções, processadores monociclo e multiciclo e desempenho trabalhados na disciplina [5]. A arquitetura multiciclo reutiliza unidades em ciclos diferentes e preserva valores intermediários em IR, A, B, ALUOut e MDR. A UC determina quais fontes, operações e escritas ficam ativas em cada estado. Essa distinção entre caminho de dados e controle orienta tanto os fluxos deste texto como os espaços preparados para as formas de onda.

1.1 Escopo e critérios do projeto

A CPU possui 32 registradores gerais de 32 bits e uma memória unificada de 256 bytes endereçada por byte. O conjunto tem 13 instruções R, oito I e duas J, catalogadas no Apêndice A. A memória lê de forma síncrona, escreve em um ciclo e armazena bytes em ordem *little-endian*. O registrador R29 atua como ponteiro da pilha (SP), com valor inicial especificado em 227. A UC também deve tratar opcode inexistente, overflow e divisão por zero, preservando em EPC o endereço da instrução causadora [1].

O Capítulo 3 da especificação exige capa, índice, introdução, unidade de processamento, descrição das entidades, descrição dos estados de controle, conjunto de simulações e conclusão. A estrutura deste relatório segue essa ordem. As exportações do circuito e do mapa de estados estão nas Figuras 2 e 6; os fluxos menores são desenhados diretamente em LaTeX para permanecerem nítidos e fáceis de atualizar.

1.2 Situação da implementação e da evidência

Os documentos disponíveis registram o desenho, as decisões de caminho de dados e os estados propostos de `push/pop`. A parte 3 acrescenta seis módulos VHDL fornecidos, o arquivo inicial de memória e onze programas Assembly de apoio [4]. Esses artefatos permitem conferir interfaces e preparar estímulos; a integração da CPU e da UC e as capturas Quartus das 23 instruções ainda precisam ser documentadas neste relatório. A Seção 5 contém um roteiro individual para cada instrução e para as entidades implementadas; os campos de resultado observado permanecem abertos para receber as evidências. As pendências técnicas de integração e orientação da disciplina aparecem no Apêndice B.

1.3 Relação com as aulas

As aulas sobre representação de instruções fundamentam a decodificação dos campos `opcode`, `rs`, `rt`, `rd`, `shamt`, `funct` e imediato. A implementação monociclo ajuda a identificar o efeito completo de uma instrução; a implementação multiciclo reparte esse efeito em etapas que compartilham ALU, memória e banco de registradores. A aula de desempenho mostra que a economia de hardware deve ser analisada junto com número de ciclos, período do clock e mistura de instruções. Portanto, a contagem de caixas em um fluxo não basta, sozinha, para concluir qual arquitetura é mais rápida [5].

2 Unidade de processamento

2.1 Visão geral do caminho de dados

O caminho de dados reúne três funções: buscar a instrução e atualizar PC; ler operandos e executar a operação; acessar a memória e escrever o resultado. A memória é compartilhada entre busca e dados. IorD escolhe PC durante a busca e ALUOut durante cargas, armazenamentos e operações de pilha. IR conserva a instrução; A e B conservam as leituras do banco; ALUOut retém endereço efetivo ou resultado; MDR retém a saída de leitura da memória.

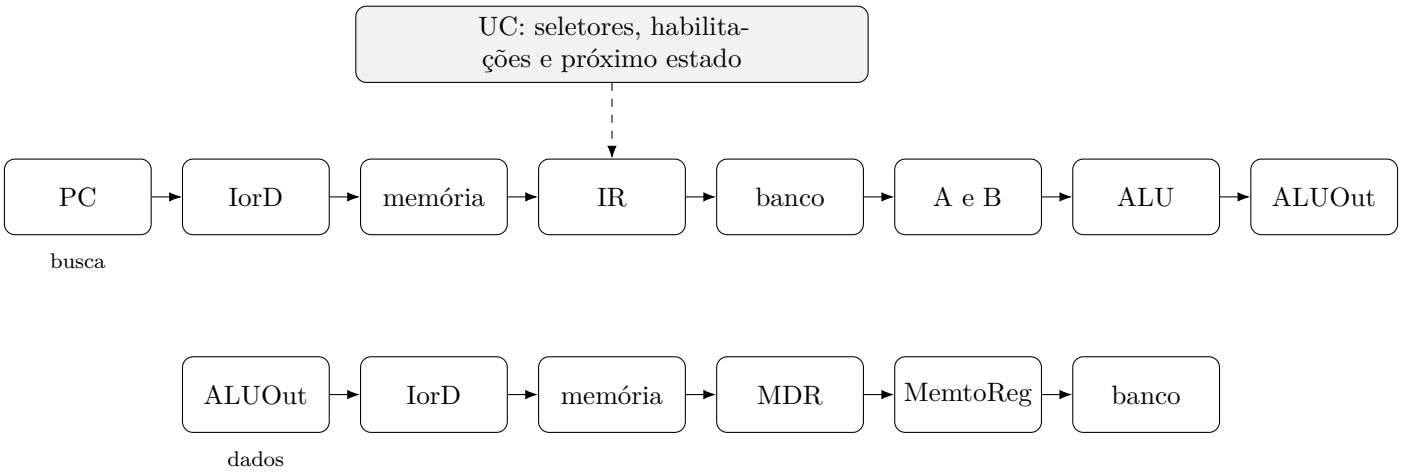


Figura 1: Fluxos funcionais da CPU multiciclo. Cada linha resume uma sequência de estados; a UC configura os caminhos a cada ciclo.

A Figura 1 serve para leitura conceitual. O desenho completo da equipe, com seletores e sinais da UC, aparece na Figura 2. O recorte seguinte amplia as conexões centrais do caminho de dados.

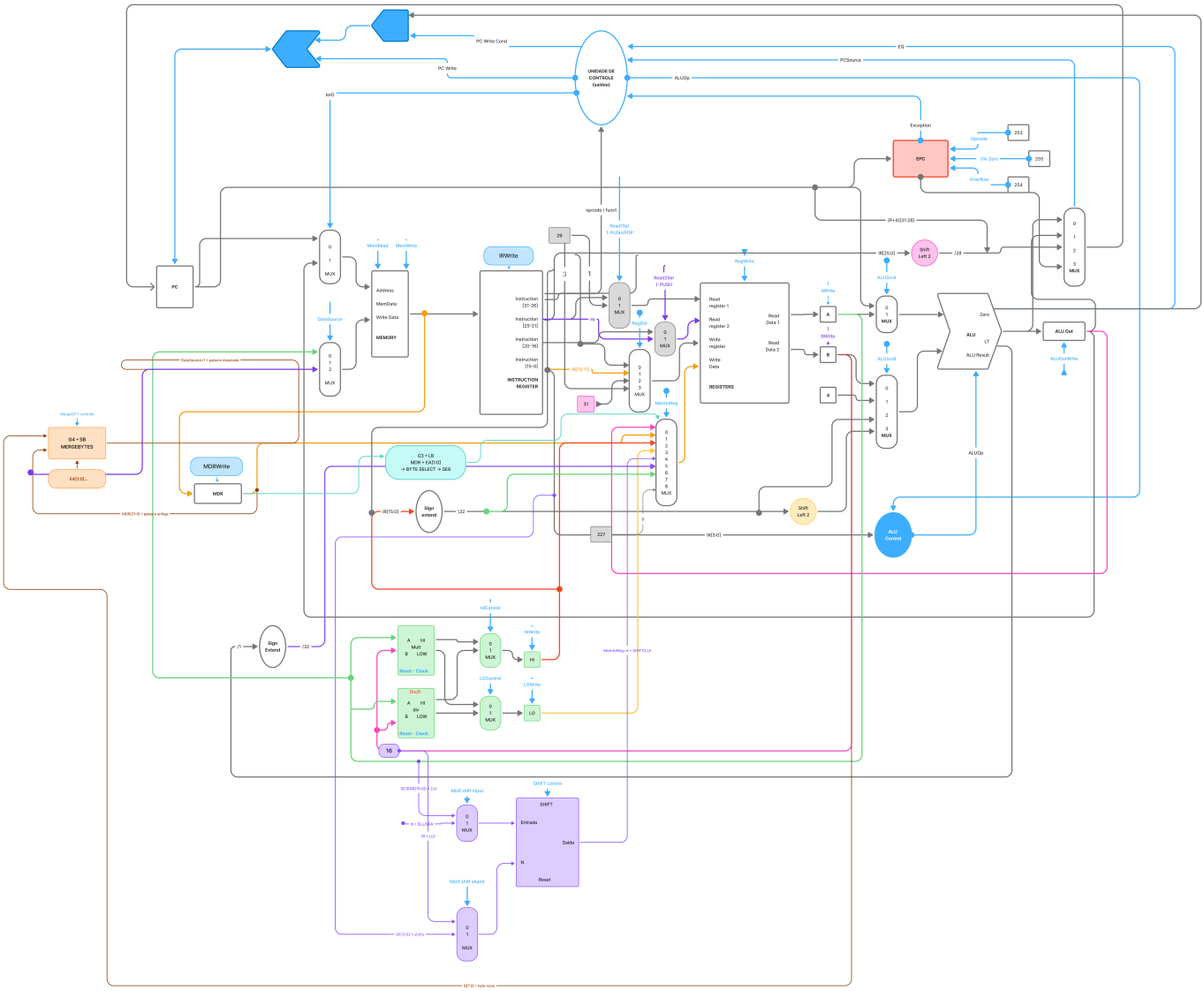


Figura 2: Circuito completo do processador multiciclo exportado do FigJam.

O recorte da Figura 3 amplia a região de busca, leitura de operandos, ALU e seleção de escrita. Ele complementa a visão integral da Figura 2, sem substituir as demais ligações do quadro.

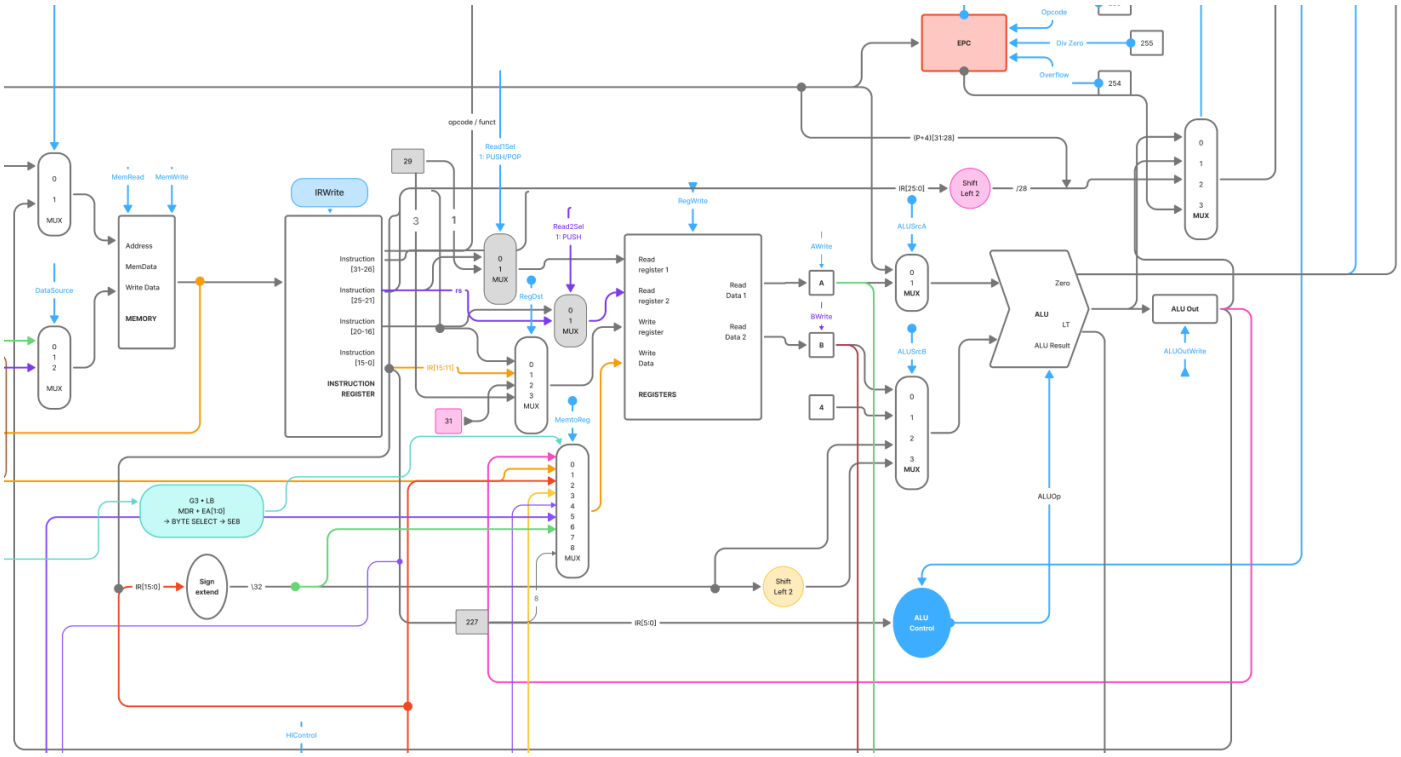


Figura 3: Detalhe do circuito: memória, IR, banco, ALU, EPC e seletores próximos.

2.2 Formatos, busca e acesso à memória

Os formatos R, I e J contêm, respectivamente, os campos `opcode|rs|rt|rd|shamt|funct`, `opcode|rs|rt|imediato` e `opcode|target`. A busca apresenta o endereço inicial P à memória e conserva a instrução em IR quando a saída estiver válida. Em paralelo ao caminho de leitura, a ALU produz $P + 4$ para o fluxo sequencial. O acesso posterior a dados usa o endereço guardado em ALUOut, sem confundir esse ciclo com a busca.

Para `lw`, `lb`, `sw` e `sb`, o endereço efetivo é

$$EA = R[rs] + \mathrm{SE}_{16}(\mathit{offset}). \tag{1}$$

Como a leitura da memória só fica disponível após a requisição, a UC deve manter endereço e IorD estáveis até o dado poder ser capturado em IR ou MDR. A escrita ocupa um ciclo. O módulo `Memoria.vhd` fornecido usa quatro bancos de oito bits e somente `Address(7 downto 0)` para os 256 bytes; a seleção e a rotação dos bancos acomodam os deslocamentos do endereço. Na ordem *little-endian*, a palavra `0xAABBCCDD` iniciada no endereço decimal 223 corresponde aos bytes `DD`, `CC`, `BB` e `AA` nos endereços 223 a 226. Assim, a estrutura do componente contempla endereço de palavra desalinhado, embora o resultado e a latência precisem ser confirmados em simulação [1, 4].

2.3 Grupos de reaproveitamento

Tabela 1: Famílias de instruções organizadas pelo caminho predominante.

Grupo	Instruções	Caminho principal
G0	Todas	PC, memória, IR e decodificação.
G1	add, and, sub, slt, addi	Banco, ALU/comparador e escrita no banco.
G2	sll, sra, lui	Fonte, SHIFT e escrita no banco.
G3	lw, lb	Endereço, memória, MDR e escrita no banco.
G4	sw, sb	Endereço, dado do banco e escrita na memória.
G5	beq, bne	Comparação e escolha condicional do PC.
G6	j, jal, jr	Formação/seleção do alvo e escrita do PC.
G7	mult, div, mfhi, mflo	Unidades especiais e registradores HI/LO.
G8	push, pop	R29, ALUOut, memória, MDR e escrita no banco.

G0 é transversal: não adiciona instruções à contagem. Os grupos G1 a G8 particionam as 23 operações para explicar o reaproveitamento, mas não substituem a decodificação por `opcode` e `funct`. Nas aulas de multiciclo, a presença de registradores intermediários permite que o mesmo recurso físico atenda grupos diferentes em ciclos sucessivos [5].

2.4 Pilha e extensão da arquitetura

O SP é R29. A pilha cresce para endereços menores. O conteúdo original do registrador fonte deve ser conservado antes de atualizar R29:

$$\texttt{push rs:}\quad d \leftarrow R[rs]_{\text{antigo}}, \qquad SP \leftarrow SP_{\text{antigo}} - 4, \qquad M_{32}[SP] \leftarrow d; \tag{2}$$

$$\texttt{pop rd:}\quad d \leftarrow M_{32}[SP_{\text{antigo}}], \qquad R[rd] \leftarrow d, \qquad SP \leftarrow SP_{\text{antigo}} + 4. \tag{3}$$

Na captura de `push`, Read1Sel seleciona R29 para A, e Read2Sel seleciona `rs` para B. Em `pop`, A recebe R29 e B não é necessário. RegDst ganha a opção R29; MemtoReg ganha a constante 227 para a proposta de inicialização; DataSource usa B como dado de escrita da memória. A largura de MemtoReg deve comportar a entrada de índice 8. Os sinais AWrite e BWrite são desabilitados depois da captura, para preservar o SP e o dado antigos durante toda a operação.

Tabela 2: Seleções essenciais da extensão da pilha.

Sinal	Seleção relevante	Finalidade
Read1Sel	1 = R29	Capturar SP antigo em A.
Read2Sel	1 = rs	Capturar dado de `push` em B.
RegDst	1 = rd; 3 = R29	Escrever destino de `pop` ou SP.
MemtoReg	0 = ALUOut; 1 = MDR; 8 = 227	Escrever endereço calculado, dado lido ou SP inicial.
DataSource	2 = B	Escrever o dado antigo de `push`; também corrige a origem de `sw`.
IorD	1 = ALUOut	Apresentar endereço de dados à memória.
ALUSrcA / ALUSrcB	A / constante 4	Calcular $SP - 4$ ou $SP + 4$.

2.5 Demais operações e cuidados de integração

`add`, `and`, `sub`, `slt` e `addi` compartilham leitura, execução e escrita no banco. `slt` deve produzir a palavra 0 ou 1, com comparação com sinal. `sll`, `sra` e `lui` compartilham SHIFT, mas variam fonte, quantidade e modo. Em `RegDesloc.vhd`, carregar a entrada e deslocá-la são comandos síncronos distintos; a UC deve prever ciclos separados ou demonstrar um caminho alternativo que produza o mesmo efeito. `lb` precisa selecionar um byte e estendê-lo com sinal; `sb` precisa preservar os três bytes vizinhos porque a memória não tem habilitação individual de byte.

`beq` e `bne` usam o alvo $P + 4 + (\mathrm{SE}_{16}(\mathit{offset}) \ll 2)$ e condições opostas. `j` e `jal` formam o alvo com os quatro bits altos do PC indicado pela especificação e `target` $\ll 2$; `jr` usa diretamente R[rs]. A documentação da equipe assume provisoriamente link $P+4$ e ausência de *delay slot* em `jal`; essa convenção exige confirmação. MULT e DIV devem ser unidades separadas da ALU. A multiplicação sinalizada usa Booth, produz 64 bits e aceita conclusão entre 32 e 40 ciclos; HI/LO guardam as metades. Na divisão, HI guarda o resto, LO o quociente e a divisão por zero comunica exceção à UC [1].

3 Descrição das entidades

Esta seção especifica a função e as interfaces relevantes dos blocos. A descrição final de cada entidade construída pela equipe deverá refletir as portas e o algoritmo do HDL efetivamente entregue, como exige a Seção 3.2.5 da especificação [1]. Componentes fornecidos pela disciplina são descritos aqui para explicar sua integração; sua simulação isolada não é exigida pelo capítulo do relatório.

3.1 Componentes recebidos na parte 3

A pasta `Componentes do Projeto - 2026.2` reúne os seis módulos da Tabela 3. O arquivo `LEIA-ME.txt` orienta incluí-los no projeto sem alterar seu código e posicionar `instrucoes.mif` junto ao arquivo `.mpf`; o conteúdo do MIF pode ser atualizado para os programas e dados dos ensaios [4]. Esses arquivos são componentes de base, e não constituem por si uma CPU integrada.

Tabela 3: Módulos VHDL e arquivo de memória fornecidos na parte 3.

Arquivo	Função observada no código	Ponto de integração
`Banco_reg.vhd`	32 registradores; duas leituras combinacionais e escrita síncrona.	Reset assíncrono zera também R29; conferir seleção e escrita de SP.
`ula32.vhd`	ALU combinacional de 32 bits com seletor de três bits.	Compatibilizar operação, flags e código da UC.
`RegDesloc.vhd`	Registro, deslocamentos lógicos e aritmético e rotações.	Carga e deslocamento usam bordas distintas.
`Registrador.vhd`	Registrador de 32 bits com habilitação de carga.	Usar para retenção temporal dos valores necessários.
`Instr_Reg.vhd`	Registra instrução e expõe campos para decodificação.	Conferir extração posterior de `rd` e `funct`.
`Memoria.vhd`	Memória em quatro bancos de bytes, com leitura em clock.	Verificar ordem dos bytes, rotação e latência dos dados.
`instrucoes.mif`	Inicialização de 256 endereços de oito bits.	Carregar programa e dados correspondentes a cada ensaio.

3.2 Registradores e banco

PC, IR, A/B, ALUOut, MDR e EPC conservam, respectivamente, o endereço de busca, a instrução, os operandos, o resultado intermediário, o dado lido e o endereço da instrução que causou exceção. As habilitações de escrita permitem reutilizar os mesmos blocos em ciclos diferentes.

O banco possui 32 registradores de 32 bits, duas leituras combinacionais e uma escrita na borda de subida. O reset assíncrono de `Banco_reg.vhd` zera também R29, o que exige conciliação com o SP inicial de 227 (Apêndice B). R31 recebe o retorno de `jal` conforme a convenção final adotada.

3.3 ALU, deslocador e unidades especiais

A ALU combinacional executa passagem de A (000), soma (001), subtração (010), AND (011), incremento (100), NOT (101) e XOR (110), além de fornecer flags de condição. A UC deve escolher o código em cada estado; `pop`, por exemplo, usa operações distintas para obter o endereço e incrementar SP. Em `RegDesloc.vhd`, carga (001), deslocamento à esquerda (010) e deslocamento aritmético à direita (100) ocorrem em bordas distintas, fato a considerar em `sll`, `sra` e `lui`.

As unidades MULT e DIV ainda dependem da implementação integrada. MULT deve produzir o produto de 64 bits por Booth com sinal; DIV deve produzir quociente e resto e sinalizar divisão por zero. HI/LO guardam os resultados, e a UC deve aguardar a conclusão antes de liberar sua leitura.

3.4 Memória e tratamento de bytes

`Memoria.vhd` usa os oito bits menos significativos de `Address` para acessar 256 bytes em quatro bancos. A rotação dos bytes e seus registros de atraso exigem conferir em forma de onda a borda correta de captura em IR e MDR, inclusive para endereços desalinhados [4].

`lb` seleciona e estende com sinal o byte endereçado. `sb` requer leitura, mescla e reescrita para preservar os outros três bytes, pois a memória não possui *byte enable*; esse caminho ainda precisa ser demonstrado no HDL e nas formas de onda.

3.5 Unidade de controle e seletores

A UC combina os campos de IR com flags, exceções e sinais de conclusão para escolher o próximo estado. Seus seletores definem as fontes do caminho de dados; suas habilitações controlam as escritas. O diagrama final deve identificar os sinais ativos em cada estado e as opções de cada multiplexador.

4 Descrição dos estados do controle

4.1 Como ler o mapa de estados

Cada estado representa uma etapa temporal da UC. Dentro de um estado, seletores e unidades combinacionais podem atuar simultaneamente; na borda, apenas os elementos habilitados capturam valores. As setas representam a sequência entre ciclos. Para explicar uma caixa do mapa, deve-se indicar: valores disponíveis no início, sinais ativados, cálculo durante o ciclo, efeito na borda e próximo estado. Essa regra, desenvolvida no guia da parte 2 [3], impede confundir ações simultâneas com estados sucessivos.

O mapa completo da Figura 6 aparece ao fim desta seção em página paisagem; a Figura 7 amplia a sequência dos estados. A legenda da Figura 5 reúne os agrupamentos de sinais. O esquema resumido da Figura 4 serve à navegação. O nome e a codificação binária dos estados desenhados devem ser conciliados com a UC implementada; aqui são usados nomes funcionais para não atribuir ao HDL estados ainda não conferidos.

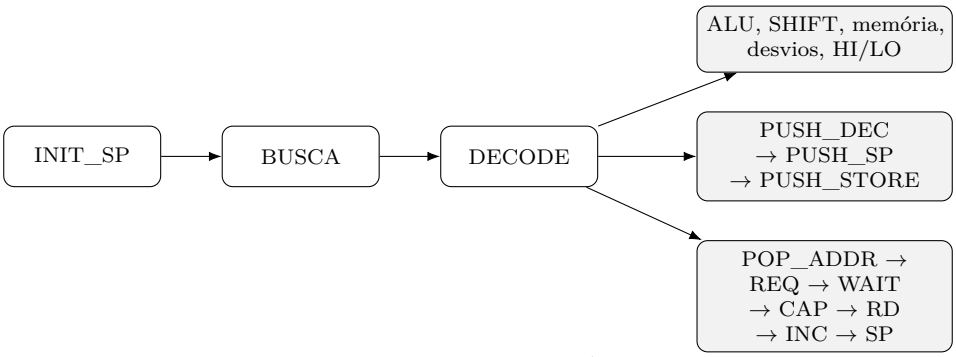


Figura 4: Resumo lógico da transição entre busca, decodificação e famílias de execução.

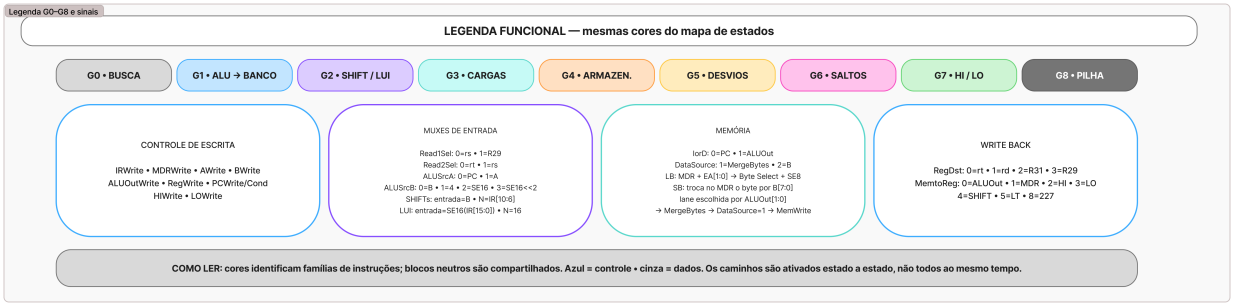


Figura 5: Legenda de grupos e sinais de controle usada no mapa de estados da equipe.

4.2 Estados compartilhados

BUSCA apresenta PC à memória, aguarda a saída síncrona quando necessário, registra IR e calcula o PC sequencial. DECODE examina `opcode` e, para instruções R, `funct`; captura operandos e escolhe a família. Estados de execução da ALU registram resultados em ALUOut e os estados de escrita escolhem destino e dado. As cargas calculam EA pela Equação 1, aguardam a leitura em MDR e escrevem no banco. Os armazenamentos apresentam EA e o dado adequado à memória. Os desvios atualizam PC somente quando sua condição se confirma; saltos escolhem o alvo formado ou R[rs]. Todos retornam à busca ao terminar.

INIT_SP é a proposta de escrita de 227 em R29 depois da liberação do reset e antes da primeira busca. Essa solução ainda deve ser confrontada com o requisito de R29=227 enquanto o reset está ativo, conforme o Apêndice B. Na captura de `push`, A recebe o SP antigo e B o dado antigo de R[rs]; na de `pop`, A recebe o SP antigo. Os seletores precisam estar estáveis antes da borda que grava A/B. Se a classificação da instrução só terminar nessa borda, acrescentar um estado específico de captura.

4.3 Sequência de `push` e `pop`

Tabela 4: Estados de `push` após a captura dos operandos.

Estado	Efeito na borda	Controles essenciais
PUSH_DEC	ALUOut $\leftarrow A - 4$	ALUSrcA=A, ALUSrcB=4, F=010, ALUOutWrite=1.
PUSH_SP	$R29 \leftarrow$ ALUOut	RegDst=3, MemtoReg=0, RegWrite=1.
PUSH_STORE	M_{32}[ALUOut] $\leftarrow B$	IorD=1, DataSource=2, wr=1.

A e B não são reescritos após a captura. Assim, `push R29` armazena o valor antigo do SP, preservado em B, apesar de R29 ter sido atualizado antes da escrita da memória. Durante esses estados, PCWrite, PCWriteCond e IRWrite permanecem desabilitados.

Tabela 5: Estados de `pop` após a captura de SP.

Estado	Efeito na borda	Controles essenciais
POP_ADDR	ALUOut $\leftarrow A$	ALUSrcA=A, F=000, ALUOutWrite=1.
POP_REQ	Requisitar leitura	IorD=1, wr=0, MDRWrite=0.
POP_WAIT	Aguardar dado válido	Endereço estável, MDRWrite=0.
POP_CAP	MDR $\leftarrow$ DataOut	IorD=1, wr=0, MDRWrite=1.
POP_RD	$R[rd] \leftarrow$ MDR	RegDst=1, MemtoReg=1, RegWrite=1.
POP_INC	ALUOut $\leftarrow A + 4$	ALUSrcA=A, ALUSrcB=4, F=001, ALUOutWrite=1.
POP_SP	$R29 \leftarrow$ ALUOut	RegDst=3, MemtoReg=0, RegWrite=1.

REQ, WAIT e CAP mantêm o mesmo endereço e representam uma leitura conservadora da memória síncrona. O `Memoria.vhd` registra o controle de rotação em dois estágios; a forma de onda da integração deve mostrar em qual borda a palavra desejada fica válida em `Dataout` e quantos estados são necessários antes de MDRWrite. A conserva SP antigo até POP_INC. POP_RD e POP_SP usam bordas diferentes porque o banco dispõe de uma única porta de escrita.

4.4 Exemplo de execução e bytes

Considere R29=227 e R8=`0xAABBCCDD`. Em `push R8`, A conserva 227, B conserva a palavra de R8, a ALU produz 223, R29 recebe 223 e a memória recebe `0xAABBCCDD` em $M_{32}[223]$. Em `pop R9`, a memória lê a partir de 223, MDR conserva a palavra, R9 a recebe e R29 volta a 227. A memória não precisa ser apagada: o avanço de SP remove o elemento logicamente da pilha.

Tabela 6: Evolução esperada no exemplo da especificação.

Momento	R29	R8	R9
Antes de `push R8`	227	`0xAABBCCDD`	—
Após `push R8`	223	`0xAABBCCDD`	—
Após `pop R9`	227	`0xAABBCCDD`	`0xAABBCCDD`

4.5 Exceções e retorno

Em opcode inexistente, overflow ou divisão por zero, a UC salva em EPC o endereço P da instrução causadora. Depois lê o byte localizado em 253, 254 ou 255, estende esse byte com zeros para 32 bits e carrega o resultado em PC. Os números 253–255 são endereços que armazenam o vetor, não o destino direto do PC. A especificação menciona `rte` como retorno $PC \leftarrow EPC$, mas não a codifica entre as 23 instruções; uma eventual implementação de `rte` deve ser decidida separadamente, sem alterar indevidamente o catálogo [1].

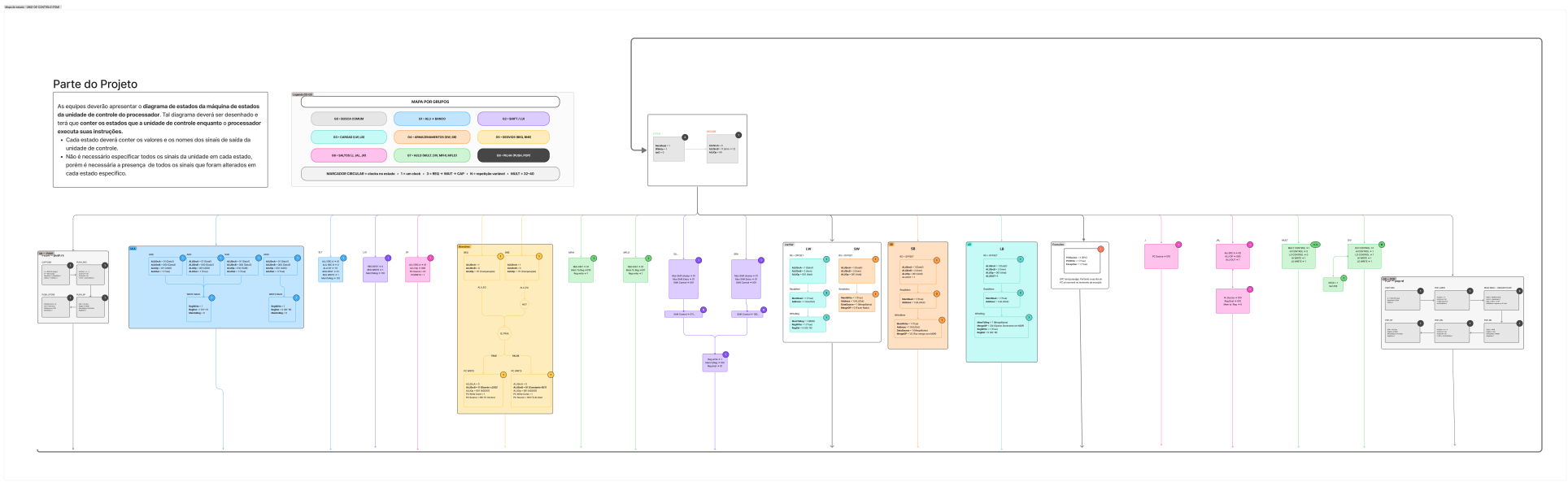


Figura 6: Visão geral do mapa de estados e do contexto de controle exportados do FigJam.

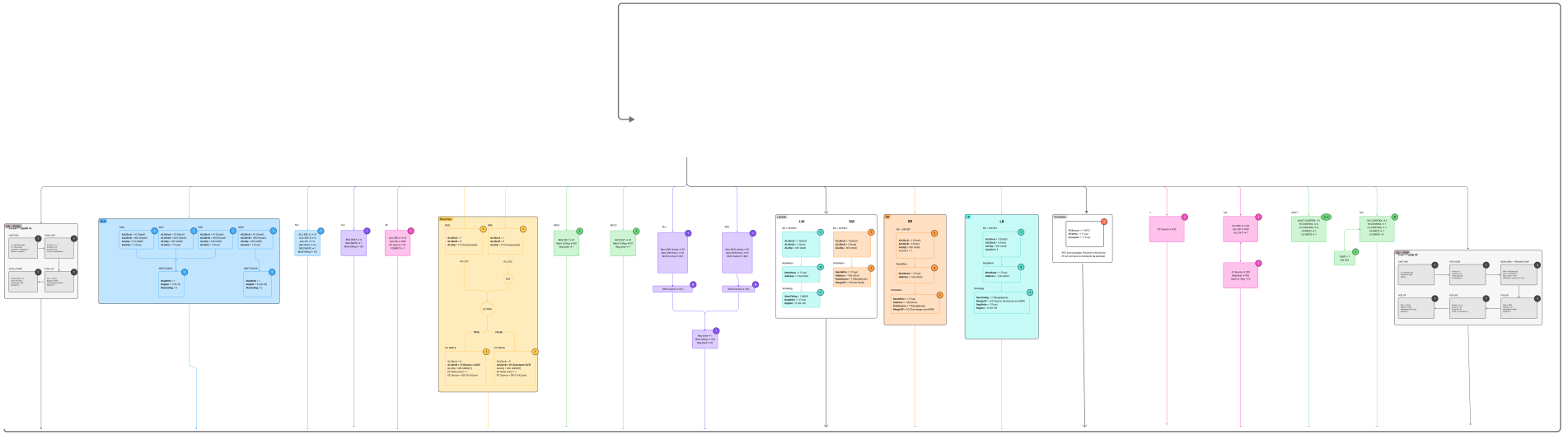


Figura 7: Recorte ampliado do mapa de estados e das transições da UC.

5 Conjunto de simulações

Carlos Henrique Gonçalves dos Santos (CC, `chgs3`) foi responsável pela parte de simulação do projeto. Esta seção está preparada para incorporar as formas de onda e a análise da implementação ao texto. A especificação exige ao menos uma simulação de cada uma das 23 instruções, com descrição dos sinais, imagem Quartus e explicação de todos os passos. Em cada captura devem aparecer clock, reset, estado da UC, PC, EPC, MDR, IR, R29, R31 e os registradores envolvidos, além dos sinais que permitem compreender a instrução [1]. Os campos *resultado observado* permanecem abertos até a inserção das evidências correspondentes.

5.1 Programas de apoio recebidos na parte 3

A pasta `mips - alunos - 2026.2` fornece os programas da Tabela 7. Eles são pontos de partida para gerar o conteúdo de `instrucoes.mif` e observar sequências reais da CPU; sua presença não equivale a resultados de simulação. O `LEIA-ME.txt` da parte 3 indica um conversor MIPS–MIF como apoio, mas cada imagem de memória deve ser conferida quanto às codificações específicas do projeto, sobretudo `push` e `pop` [4].

Tabela 7: Programas Assembly fornecidos e finalidade principal.

Arquivo	Situações que pode exercitar
`ula.asm`, `addi_overflow.asm`	Aritmética, lógica, comparação e overflow de `addi`.
`branch.asm`, `jumps_break.asm`	`beq`/`bne` tomados e não tomados; `j`, `jal` e `jr`.
`loads.asm`, `stores.asm`	`lw`/`lb` e `sw`/`sb` com dados em memória.
`shifts.asm`	`sll` e `sra`.
`mult.asm`, `div.asm`	Produto, quociente, resto, HI/LO e divisão por zero.
`push_pop.asm`	Um `push` seguido de um `pop`; complementar com LIFO e `push R29`.
`excecoes.asm`	Cenários de divisão, overflow e opcode não reconhecido; validar a montagem do último estímulo.

Os programas não substituem a exigência de uma captura e de uma análise para cada uma das 23 instruções. Também faltam, como estímulos explícitos, os casos de byte em todas as posições, palavra no endereço 223, pilha com duas entradas, limites de deslocamento e temporização de cada exceção. Esses cenários devem ser acrescentados pela equipe durante a validação.

5.2 Convenção de preenchimento

Para cada instrução: (1) registrar valores iniciais e programa de teste; (2) definir sinais visíveis e a escala temporal; (3) inserir uma captura legível; (4) percorrer os estados na ordem real, indicando o valor antes e depois de cada borda; (5) comparar valores medidos com os esperados; (6) declarar aprovação ou falha com causa conhecida. Os nomes das capturas podem ser inseridos diretamente no campo de figura de cada subseção ou associados a novos arquivos na pasta `figuras/`. Não usar o mapa de estados como substituto de uma forma de onda.

5.3 Casos prioritários da pilha

Tabela 8: Testes mínimos sugeridos para a extensão da pilha.

Caso	Preparação	Resultado esperado
Inicialização	Liberar reset e entrar em INIT_SP.	R29=227 antes da primeira busca.
`push R8`	R29=227; R8=`0xAABBCCDD`.	R29=223; M_{32}[223] recebe a palavra.
`pop R9`	R29=223; M_{32}[223] contém a palavra.	R9 recebe a palavra; R29=227.
LIFO	Dois `push`, seguidos de dois `pop`.	SP: 227, 223, 219, 223, 227; ordem invertida dos dados.
`push R29`	R29=227.	Memória em 223 recebe 227, não 223.
Leitura síncrona	Observar REQ, WAIT e CAP.	Endereço estável; MDRWrite apenas em CAP.
Little-endian	Escrever `0x12345678` em 223.	Bytes 223–226: `78 56 34 12`.

5.4 Instruções do formato R

5.5 add

Objetivo: Verificar soma com sinal, escrita em rd e tratamento de overflow.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rd recebe a soma válida; em overflow, a exceção segue a política da UC.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para add.

Figura 8: Simulação de add — inserir captura após validação.

5.6 and

Objetivo: Verificar a operação lógica bit a bit.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rd recebe a conjunção dos 32 pares de bits.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para and.

Figura 9: Simulação de and — inserir captura após validação.

5.7 div

Objetivo: Verificar quociente, resto, conclusão e divisão por zero.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: LO recebe quociente, HI recebe resto; divisor zero sinaliza exceção.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para div.

Figura 10: Simulação de div — inserir captura após validação.

5.8 mult

Objetivo: Verificar Booth com sinal, produto de 64 bits e latência.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: HI/LO formam o produto; conclusão ocorre dentro do limite da especificação.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para mult.

Figura 11: Simulação de mult — inserir captura após validação.

5.9 jr

Objetivo: Verificar salto por registrador.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: PC recebe o conteúdo de R[rs].

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

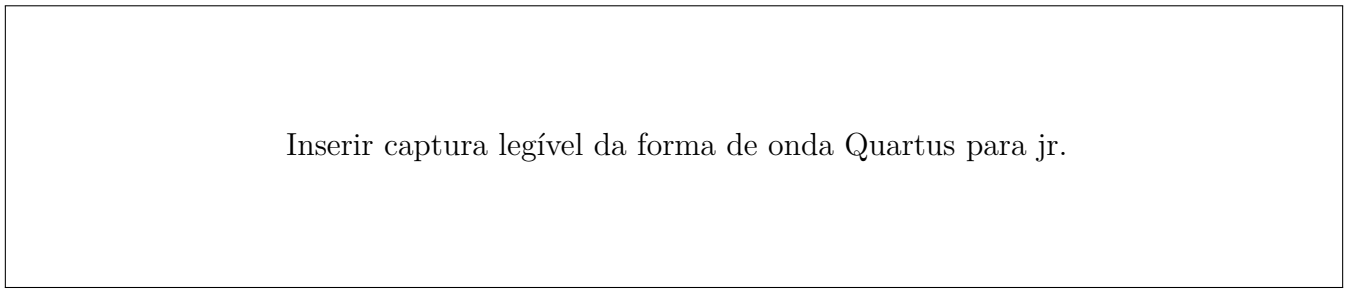


Figura 12: Simulação de jr — inserir captura após validação.

5.10 mfhi

Objetivo: Verificar leitura de HI sem alterá-lo.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rd recebe o valor armazenado em HI.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

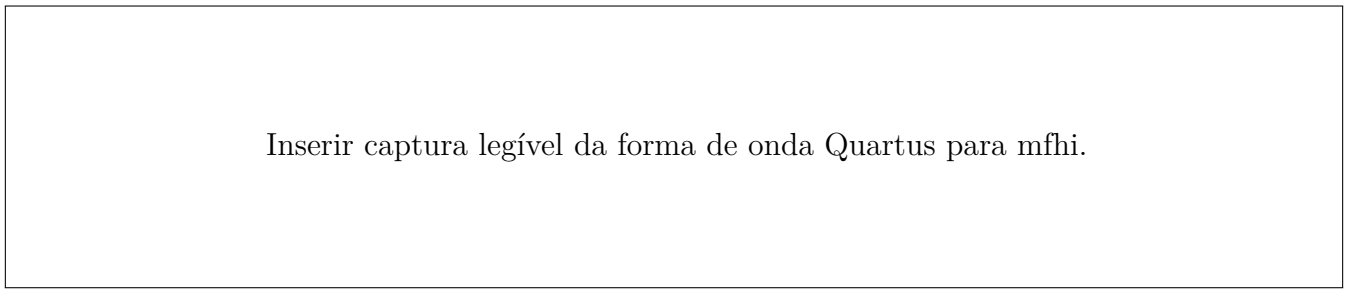


Figura 13: Simulação de mfhi — inserir captura após validação.

5.11 mflo

Objetivo: Verificar leitura de LO sem alterá-lo.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rd recebe o valor armazenado em LO.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para mflo.

Figura 14: Simulação de mflo — inserir captura após validação.

5.12 sll

Objetivo: Verificar deslocamento lógico, inclusive quantidades 0 e 31.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rd recebe R[rt] deslocado à esquerda, com zeros inseridos.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para sll.

Figura 15: Simulação de sll — inserir captura após validação.

5.13 slt

Objetivo: Verificar comparação com sinal e extremos.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rd recebe exatamente 0 ou 1.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para slt.

Figura 16: Simulação de slt — inserir captura após validação.

5.14 sra

Objetivo: Verificar deslocamento aritmético e preservação do sinal.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rd recebe R[rt] deslocado à direita com replicação do bit de sinal.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para sra.

Figura 17: Simulação de sra — inserir captura após validação.

5.15 sub

Objetivo: Verificar subtração com sinal e overflow.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rd recebe a diferença válida; overflow desvia ao tratamento.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para sub.

Figura 18: Simulação de sub — inserir captura após validação.

5.16 push

Objetivo: Verificar captura de R[rs] antigo, decremento de SP e escrita da palavra.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: SP diminui quatro; a memória recebe o valor preservado em B.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

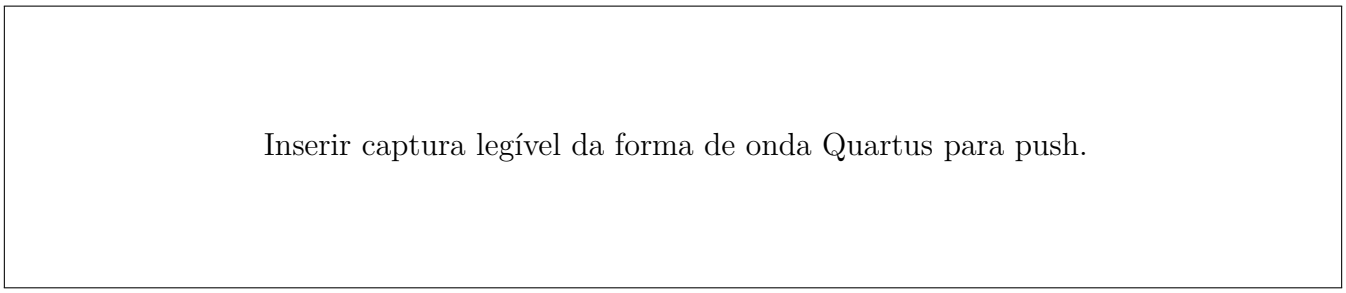


Figura 19: Simulação de push — inserir captura após validação.

5.17 pop

Objetivo: Verificar leitura síncrona, escrita de rd e incremento posterior de SP.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rd recebe a palavra do topo antigo; R29 aumenta quatro.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

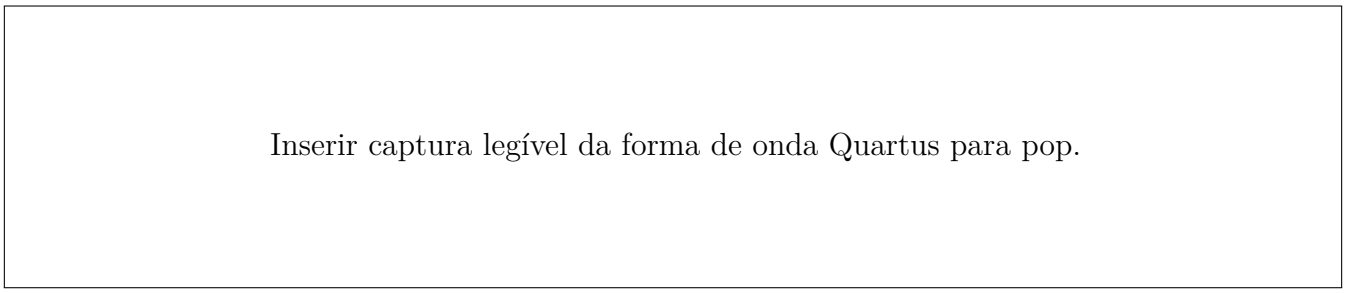


Figura 20: Simulação de pop — inserir captura após validação.

5.18 Instruções do formato I

5.19 addi

Objetivo: Verificar extensão de sinal do imediato e soma.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rt recebe a soma válida; overflow é tratado.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para addi.

Figura 21: Simulação de addi — inserir captura após validação.

5.20 beq

Objetivo: Verificar condições verdadeira/falsa e offset positivo/negativo.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: PC recebe o alvo relativo a P+4 somente quando R[rs]=R[rt].

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para beq.

Figura 22: Simulação de beq — inserir captura após validação.

5.21 bne

Objetivo: Verificar a inversão da condição de igualdade.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: PC recebe o alvo somente quando R[rs] difere de R[rt].

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para bne.

Figura 23: Simulação de bne — inserir captura após validação.

5.22 lb

Objetivo: Verificar as quatro posições do byte e extensão de sinal.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rt recebe o byte endereçado estendido para 32 bits.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para lb.

Figura 24: Simulação de lb — inserir captura após validação.

5.23 lui

Objetivo: Verificar seleção do imediato e deslocamento de 16 posições.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rt recebe imediato nos 16 bits altos e zeros nos baixos.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para lui.

Figura 25: Simulação de lui — inserir captura após validação.

5.24 lw

Objetivo: Verificar endereço efetivo, latência da memória e MDR.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: rt recebe a palavra lida após a captura válida.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para lw.

Figura 26: Simulação de lw — inserir captura após validação.

5.25 sb

Objetivo: Verificar escrita de byte sem perda dos vizinhos.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: Apenas o byte endereçado muda na palavra da memória.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para sb.

Figura 27: Simulação de sb — inserir captura após validação.

5.26 sw

Objetivo: Verificar que o dado gravado vem de R[rt] via B.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: A palavra em EA recebe o valor de R[rt].

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para sw.

Figura 28: Simulação de sw — inserir captura após validação.

5.27 Instruções do formato J

5.28 j

Objetivo: Verificar formação do alvo e atualização do PC.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: PC recebe o destino formado pelos bits altos e target deslocado.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para j.

Figura 29: Simulação de j — inserir captura após validação.

5.29 jal

Objetivo: Verificar alvo e endereço de retorno após definir a convenção.

Preparação e programa de teste: *Inserir valores iniciais, palavra de instrução e sequência usada.*

Sinais observados: clock, reset, estado da UC, PC, EPC, IR, MDR, R29, R31 e os registradores e sinais específicos da instrução.

Resultado esperado: PC recebe o alvo; R31 recebe o link acordado para esta CPU.

Resultado observado e análise ciclo a ciclo: *Inserir os dados da simulação realizada. Descrever estados, valores antes e depois das bordas e comparação com o resultado esperado.*

Inserir captura legível da forma de onda Quartus para jal.

Figura 30: Simulação de jal — inserir captura após validação.

5.30 Simulações isoladas das entidades implementadas

A especificação dispensa simulação isolada dos componentes fornecidos e dos multiplexadores, mas exige imagem e explicação para cada unidade construída pela equipe. Para cada entidade implementada, duplicar o modelo abaixo e preencher nome, portas, estímulos, resultado esperado, resultado observado e figura.

5.30.1 Unidade de controle

Entradas e saídas: *Preencher de acordo com o HDL.*
Cenários: decodificação das 23 instruções, transições de retorno à busca, reset, exceções e estados da pilha.
Forma de onda e análise: *Inserir captura e explicar cada transição relevante.*

5.30.2 Multiplicador

Portas e algoritmo: *Preencher conforme a unidade implementada.*
Cenários: sinais dos operandos, produto de 64 bits, início, conclusão, 32–40 ciclos e reset interno.
Forma de onda e análise: *Inserir captura e conferir HI/LO.*

5.30.3 Divisor

Portas e algoritmo: *Preencher conforme a unidade implementada.*
Cenários: quociente, resto, sinais, divisor zero, conclusão e reset interno.
Forma de onda e análise: *Inserir captura e conferir HI/LO.*

5.30.4 Outras entidades próprias

Entidades a incluir: *Listar apenas as efetivamente implementadas pela equipe, por exemplo MergeBytes, comparador ou lógica adicional de exceções. Criar uma subsubseção e figura para cada uma.*

6 Conclusão

O projeto documental organiza uma CPU multiciclo para as 23 instruções da especificação 2026.2 e explicita como a extensão de pilha reutiliza o banco, a ALU e a memória. `push` conserva o operando antes de atualizar R29; `pop` lê a palavra no SP antigo antes de incrementar o ponteiro. A sequência proposta reproduz o exemplo $227 \rightarrow 223 \rightarrow 227$ e reserva ciclos separados para a leitura síncrona e as duas escritas do `pop`.

A parte 3 disponibiliza componentes VHDL, inicialização da memória e programas Assembly que tornam mais precisa a análise das interfaces e do plano de testes. As imagens do circuito e do mapa de estados já foram incorporadas. A avaliação do funcionamento efetivo no relatório ainda depende de documentar a integração da CPU e da UC e inserir as capturas e análises das 23 instruções e das entidades construídas pela equipe. As subseções da Seção 5 foram preparadas para receber esses dados sem confundir expectativa com medição. Antes da entrega, a equipe deve resolver as pendências do Apêndice B e conferir as figuras contra a implementação final.

Referências

[1] UNIVERSIDADE FEDERAL DE PERNAMBUCO. *Infra-Estrutura de Hardware: Especificação de Projeto 2026.2*. Recife, 2026. Arquivo local: `Projeto/Especificacao Projeto 2026-2.pdf`.

[2] EQUIPE EQ01. *Documentação PT.1: análise, catálogo de instruções, arquitetura, estados, decisões e validação*. Pasta `Projeto/Documentacao PT.1`, 2026.

[3] EQUIPE EQ01. *Documentação PT.2: roteiro e guia lógico do mapa de estados*. Pasta `Projeto/Documentacao PT.2`, 2026.

[4] IF674. *Documentação PT.3: componentes VHDL, arquivo inicial da memória e programas Assembly de teste*. Pasta `Projeto/Documentação PT.3`, 2026.

[5] IF674. *Slides consolidados de instruções, processador monociclo, processador multiciclo e desempenho*. Pasta `Aulas/Slides consolidados`, 2026.

[6] EQUIPE EQ01. *Projeto IH 2026.2 — quadro do circuito e mapa de estados*. FigJam. `https://www.figma.com/board/hILCPGWNEfBjOQvYNMzxR0/ProjetoIH-2K26.2--EQ01?node-id=0-1`. Acesso em outubro de 2026.

A Catálogo das 23 instruções

As codificações seguem as Tabelas 1–3 da especificação 2026.2. Em todas as instruções R, `opcode=0x00`. `push` e `pop` são extensões desta disciplina e podem não ser aceitas por montadores MIPS genéricos.

Tabela 9: Instruções exigidas e efeito arquitetural principal.

Formato	Instrução	Código	Efeito principal
R	`add rd,rs,rt`	funct 0x20	Soma com sinal, destino rd e controle de overflow.
R	`and rd,rs,rt`	funct 0x24	AND bit a bit em rd.
R	`div rs,rt`	funct 0x1A	Quociente em LO, resto em HI, erro de divisor zero.
R	`mult rs,rt`	funct 0x18	Produto com sinal de 64 bits em HI/LO.
R	`jr rs`	funct 0x08	PC recebe R[rs].
R	`mfhi rd`	funct 0x10	rd recebe HI.
R	`mflo rd`	funct 0x12	rd recebe LO.
R	`sll rd,rt,shamt`	funct 0x00	Deslocamento lógico à esquerda.
R	`slt rd,rs,rt`	funct 0x2A	Comparação com sinal; rd recebe 0 ou 1.
R	`sra rd,rt,shamt`	funct 0x03	Deslocamento aritmético à direita.
R	`sub rd,rs,rt`	funct 0x22	Subtração com sinal e controle de overflow.
R	`push rs`	funct 0x05	R29 diminui quatro; R[rs] antigo é gravado no novo topo.
R	`pop rd`	funct 0x06	rd recebe a palavra do topo antigo; R29 aumenta quatro.
I	`addi rt,rs,imm`	opcode 0x08	Soma com imediato estendido com sinal.
I	`beq rs,rt,off`	opcode 0x04	Desvio relativo se R[rs]=R[rt].
I	`bne rs,rt,off`	opcode 0x05	Desvio relativo se R[rs]≠R[rt].
I	`lb rt,off(rs)`	opcode 0x20	Leitura de byte e extensão de sinal.
I	`lui rt,imm`	opcode 0x0F	Imediato nos 16 bits altos.
I	`lw rt,off(rs)`	opcode 0x23	Leitura de palavra em endereço efetivo.
I	`sb rt,off(rs)`	opcode 0x28	Escrita de byte com preservação dos vizinhos.
I	`sw rt,off(rs)`	opcode 0x2B	Escrita da palavra de R[rt] em endereço efetivo.
J	`j target`	opcode 0x02	Salto incondicional para alvo formado.
J	`jal target`	opcode 0x03	Salto com gravação de endereço de retorno em R31.

B Decisões a confirmar antes da entrega

Os itens abaixo separam requisitos explícitos, convenções provisórias e verificações que só podem ser encerradas com o HDL ou com orientação da disciplina. Sua presença evita que o relatório atribua certeza a comportamentos ainda não demonstrados.

Tabela 10: Pendências de integração e validação.

ID	Tema	Conferência necessária
D01	`jal`	Confirmar link e existência de *delay slot*. A documentação assume provisoriamente $P + 4$, sem *delay slot*.
D02	Reset de R29	Conciliar exigência de SP=227 durante reset com o banco fornecido, cujo reset assíncrono limpa os registradores. INIT_SP só escreve após a liberação.
D03	Endereço 223	A rotação dos bancos de `Memoria.vhd` prevê endereço desalinhado; demonstrar em simulação que a palavra em 223 usa exatamente os bytes 223–226.
D04	`pop R29`	Definir a prioridade entre a escrita do dado lido em rd=29 e a atualização final de SP. A sequência proposta faz o incremento prevalecer.
D05	Divisão extrema	Definir comportamento de `0x80000000`/-1, cujo quociente não cabe em 32 bits com sinal.
D06	`rte`	A especificação menciona retorno via EPC, mas não fornece sua codificação entre as 23 instruções.
D07	`lb`/`slt`	Confirmar extensão de sinal de byte e formação de 0/1 em 32 bits para comparação com sinal.
D08	Latência de memória	Confrontar REQ/WAIT/CAP com a temporização real da memória e de MDR antes de otimizar estados.
D09	Outras instruções	Conferir origem B de `sw`, condição invertida de `bne`, bytes de `sb`, alvo de saltos, conclusão de MULT/DIV e vetores de exceção.
D10	Integração da parte 3	Integrar os módulos fornecidos sem modificar seu código, completar CPU, UC, MULT/DIV e lógica de bytes; conferir portas e sinais reais.
D11	Programas de teste	Gerar o MIF dos programas Assembly e complementar os estímulos para cobrir as 23 instruções, LIFO, endereço 223 e exceções.